

AUTOR(A) DA DISSERTAÇÃO

Título da Dissertação



UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE MATEMÁTICA

2019

AUTOR(A) DA DISSERTAÇÃO

Título da Dissertação

Dissertação apresentada ao Programa de Pós-
Graduação...

UBERLÂNDIA - MG
2010

Dedicatória.

Agradecimientos

Agradecimientos.

Resumo

Resumo em português.

foi quando a vida me deu um doce

Lista de Figuras

1.1	Estrutura de um neurônio típico.	14
1.2	Modelo de uma unidade de McCulloch e Pitts.	15
1.3	Gráficos das Funções de Ativação.	16
1.4	Representação da taxa de aprendizagem.	19
1.5	Estrutura visual básica de uma unidade e uma rede neural com duas camadas ocultas.	20
1.6	Arquiteturas de Redes Neurais.	22
1.7	Ilustração do processo de backpropagation em uma rede neural.	23
1.8	Comparação entre o método de descida do gradiente tradicional e o Gradiente Descendente Estocástico.	25
1.9	Ilustração do Overfitting.	25
1.10	Ilustração do Dropout removendo 60% das unidades aleatoriamente durante uma época de treinamento.	27
1.11	Ilustração do Early Stopping.	28
1.12	Gráfico da derivada da função sigmoid logística.	29
1.13	Exemplo de uma imagem/matriz 10×10 em escala de cinza, onde cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco).	31
1.14	Exemplos de dígitos manuscritos do conjunto de dados MNIST.	32
1.15	Arquitetura da rede neural para classificação de dígitos manuscritos. É enviado o número de pixels da imagem (784) para a camada de entrada (unidade $x_{784} = 0,4$ intensidade do pixel), que é processada por uma camada oculta com 30 unidades, e a camada de saída tem 9 unidades, correspondendo às 10 classes de dígitos (0 a 9).	32
1.16	Fluxo de dados durante o processo de treinamento e avaliação da rede neural.	33
1.17	Exemplo de convolução bidimensional. A função $X(t)$ é convoluída com o <i>kernel</i> $W(a)$ para produzir a saída $S(t)$	36

- 1.18 Aplicações de diferentes *kernels* em uma imagem. A imagem original é representada em (a), juntamente com o *kernel* identidade, que não faz modificações. Em (b) a imagem passou por um *kernel* que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um *kernel* de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um *kernel* que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O *kernel* em (f) reforça os contornos (bordas) da imagem. . . . 37
- 1.19 Aplicação de padding em uma imagem. Em (a) a imagem original passou por zero padding. Em (b) a imagem original passou por *reflection padding*. A imagem em (c) passou por um padding constante, semelhante em (a), mas com valor diferente de zero. 39
- 1.20 Exemplo de Max Pooling com tamanho de 2x2. A imagem original é reduzida para a imagem menor após a aplicação do Max Pooling. Cada bloco de 2x2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida. 39
- 1.21 Convolução transposta. Uma convolução de *kernel* 3x3 com stride 1 em uma imagem 5×5 geraria uma imagem de dimensões 3×3 . A convolução transposta aplicada na imagem 3×3 (vermelha) para se obter uma imagem 5×5 (verde) é feita em duas etapas. Primeiramente (centro) se insere pixels de valor 0, representados pela cor cinza, entre os pixels da imagem original, juntamente com pixels de borda. Em seguida se faz uma convolução nessa imagem alterada também usando um *kernel* 3x3 e stride 1. 41
- 1.22 Arquitetura típica de uma Rede Neural Convolutacional com uma camada de convolução e uma camada de pooling, em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem. 41
- 1.23 Arquitetura da U-Net, composta por uma parte de contração (encoder) e uma parte de expansão (decoder). O encoder extrai características da imagem de entrada, enquanto o decoder reconstrói a imagem segmentada. 43

Lista de Tabelas

1.1	Banco de dados com as avaliações dos colegas e a decisão final de compra.	17
-----	---	----

Sumário

Agradecimentos	4
Resumo	5
Lista de Figuras	8
Lista de Tabelas	9
Introdução	12
1 Fundamentação Teórica	13
1.0.1 Unidade/Neurônio	13
1.0.2 Neurônio Biológico	13
1.0.3 Unidade	14
1.0.4 Tipos de Função de Ativação	15
1.0.5 Perceptron e Algoritmo de Aprendizagem	16
1.1 Redes Neurais	19
1.1.1 Arquiteturas de Redes Neurais	20
1.1.2 Backpropagation	21
1.1.3 Gradiente Descendente Estocástico	24
1.2 Overfitting	24
1.2.1 Hiperparâmetros	26
1.2.2 Regularização	26
1.2.3 Momentum	28
1.2.4 Vanishing Gradient	29
1.2.5 Córtex Visual Humano e Visão Computacional	30
1.2.6 Dígitos Manuscritos	31
1.3 Redes Neurais Convolucionais	34
1.3.1 Convolução	34
1.3.2 <i>Kernel</i> Multicamadas	38
1.3.3 Padding	38
1.3.4 Pooling	39
1.3.5 Stride	40

1.3.6	Convolução Transposta	40
1.4	U-net	42
1.4.1	Copy and Crop	42

Introdução

Helloo baby

Capítulo 1

Fundamentação Teórica

1.0.1 Unidade/Neurônio

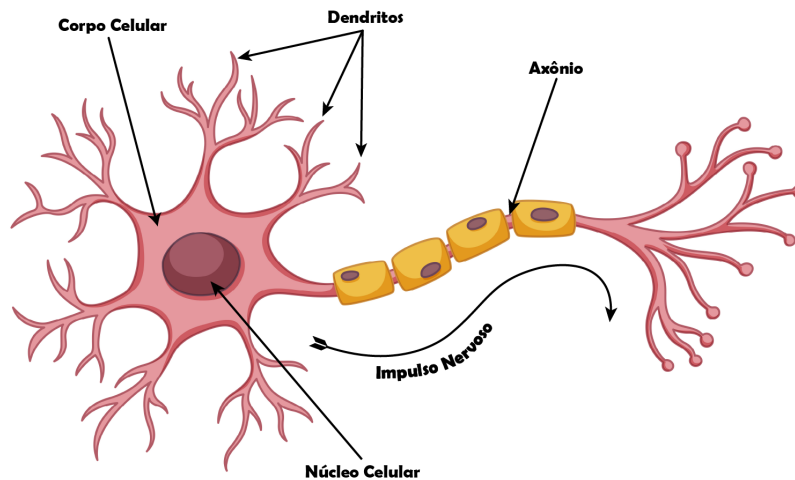
Na área de Inteligência Artificial, comumente utiliza-se o termo *neurônio artificial* para se referir a uma unidade de processamento em um modelo de rede neural. O neurônio artificial é inspirado no funcionamento do neurônio biológico, que é a unidade básica do sistema nervoso. Assim como os neurônios biológicos, os neurônios artificiais recebem entradas, processam essas entradas e produzem uma saída. A principal função de um neurônio artificial é realizar uma combinação linear das entradas, seguida por uma função de ativação não linear, que determina a saída do neurônio. A semelhança entre esses dois objetos (Neurônio artificial/Neurônio Biológico) só está na inspiração, sendo bem diferentes em seu funcionamento, com isso, neste trabalho utiliza-se o termo *unidade* para se referir ao neurônio artificial.

1.0.2 Neurônio Biológico

Para entender o funcionamento da unidade matemática é necessário entender o funcionamento do neurônio biológico. Neurônios são “células excitáveis” que se comunicam ao transmitir impulsos elétricos. Células *excitáveis* são definidas como células capazes de produzir sinais elétricos amplos e rápidos denominados *potências de ação* (veja [Stanfield 2014]). A maioria dos neurônios contém três componentes principais: um corpo celular, o(s) dendrito(s) e um axônio. O corpo celular, chamado de *soma* contém o núcleo celular, local onde parte da informação recebida de outras partes do corpo é manipulada. Os *dendritos* se ramificam a partir do corpo celular, são responsáveis por receber e integrar uma imensa quantidade de sinais provenientes de outros neurônios. Outra ramificação advinda do corpo celular é o *axônio*. Diferente dos dendritos, cuja função é *receber* informação, um axônio *envia* informações. Grande parte dos neurônios tem apenas um axônio, mas os axônios conseguem se ramificar, enviando, assim, sinais a mais de um destino. Esses três componentes, de forma resumida, formam o neurônio biológico, como mostrado na Figura 1.1.

Outro fator importante em um neurônio biológico são os potenciais de ação, gerados nos dendritos, são avaliados e ponderados por meio da *força sináptica* [Bear, Connors e Paradiso 2017], que nos diz o quão importante a informação. Após esse processo, são enviados para o núcleo

Figura 1.1: Estrutura de um neurônio típico.



Fonte: Adaptado de [Stanfield 2014].

celular. Entre o corpo celular e o axônio será decidido se o potencial é excitatório ou inibitório. A decisão é feita caso o potencial de ação supere ou não um *limiar*. Caso seja excitatório, a informação é passada a diante, caso seja inibitório, a informação é suprimida.

1.0.3 Unidade

A partir das informações obtidas com o estudo do neurônio biológico, pesquisadores tentaram simular seu funcionamento em computador. A implementação mais bem aceita de simulação foi proposta por dois pesquisadores, o neurobiologista Warren McCulloch e o matematico Walter Pitts em 1943 em [McCulloch 1943] que utilizaram-se da lógica proposicional para descrever o funcionamento do sistema nervoso central, Figura 1.2. McCulloch acreditava que o neurônio bilógico tinha uma natureza binária, ou seja, um neurônio influencia outro ou não, entretanto, posteriormente essa crença mostrou-se errônea com os avanços de estudos na área.

As variáveis $x_1, x_2, x_3, \dots, x_n$ são denominadas *dados de entrada*, em analogia às sinapses de um neurônio biológico. Os valores $w_1, w_2, w_3, \dots, w_n$ são os *pesos* associados a cada dado de entrada, os pesos nos dizem o quão importante é a entrada para a *soma*, parte central em laranja na Figura 1.2. O viés tem a mesma funcionalidade do limiar no neurônio biológico, determinando se o neurônio será ativado ou não. Para resumir a notação, defini-se o viés de b (do inglês, *bias*). Na soma, a variável z é obtida fazendo a combinação linear das variáveis:

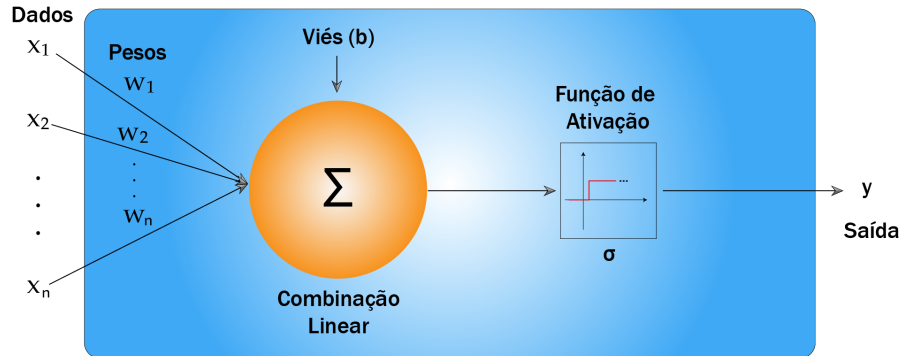
$$x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + \dots + x_n * w_n + b = z = \sum_{i=1}^n w_i x_i + b.$$

Em seguida, o valor z é aplicado a uma *função de ativação* $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, resultando na saída $\hat{y}$, ou

seja:

$$\hat{y} = \sigma \left(\sum_{i=1}^n w_i x_i + b \right). \quad (1.1)$$

Figura 1.2: Modelo de uma unidade de McCulloch e Pitts.



Fonte: Elaboração do Autor.

1.0.4 Tipos de Função de Ativação

A função de ativação, representada por σ , define a saída de uma unidade, pode-se definir diferentes tipos de funções de ativação, como a função de limiar, função linear por partes e a função sigmoideal.

- *Função de Limiar*: Para este tipo de função de ativação, descrito na Equação (1.2), a unidade é ativada (ou seja, produz uma saída de 1) se a soma ponderada das entradas mais o viés for maior ou igual a zero. Caso contrário, a saída é 0.

$$\sigma(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}. \quad (1.2)$$

- *Função Sigmoideal*: A função sigmoideal, cujo o gráfico tem a forma de um “S”, é de longe a forma mais comum de função de ativação utilizada na construção de unidades. Esta é definida como uma função estritamente crescente que exibe um balanceamento adequado entre comportamento linear e não-linear. Um exemplo de função sigmoideal é a *sigmoid logística*, definida por:

$$\sigma(z) = \frac{1}{1 + \exp(-\alpha z)}, \quad (1.3)$$

em que α é um parâmetro que controla a inclinação da curva. A função sigmoid logística tem um conjunto de propriedades que se mostrarão muito úteis nos cálculos relacionados à aprendizagem dos pesos:

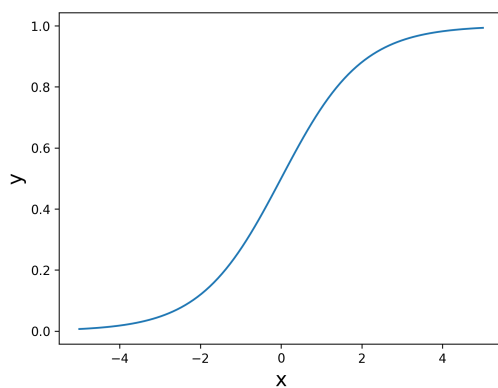
1. Não linear;

2. Contínua e diferenciável em todo o domínio de $\mathbb{R}$;
 3. A derivada tem forma simples e é expressa pela própria função: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$;
 4. É estritamente monótona, ou seja, $z_1 \leq z_2 \iff \sigma(z_1) \leq \sigma(z_2)$.
- *Função Rectified Linear Unit (ReLU)*: A função ReLU é definida como $\sigma(z) = \max(0, z)$. Esta é amplamente utilizada em redes neurais profundas (estudada na seção 1.1) devido à sua simplicidade computacional. A função ReLU ativa a unidade apenas quando a soma ponderada das entradas mais o viés é positiva, caso contrário, a saída é zero, ou seja:

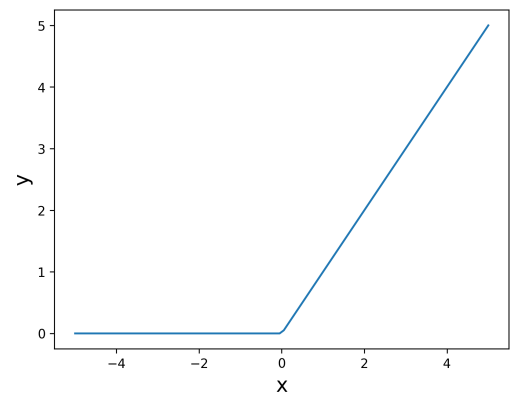
$$\sigma(z) = \begin{cases} z, & \text{se } z > 0 \\ 0, & \text{se } z \leq 0 \end{cases}. \quad (1.4)$$

Em geral, o objetivo da função de ativação é limitar a saída de uma unidade para um intervalo de valores, a Figura 1.3a mostra o gráfico da função sigmoid logística, enquanto a Figura 1.3b mostra o gráfico da função *Rectified Linear Unit*.

Figura 1.3: Gráficos das Funções de Ativação.



(a) Função Sigmoid.



(b) Função ReLU.

Fonte: Elaboração do Autor.

A principal desvantagem na unidade de McCulloch e Pitts é a ausência de um algoritmo de aprendizagem, ou seja, uma maneira de ajustar os pesos e o viés da unidade para que ele seja capaz de resolver um problema específico. Essa limitação foi superada em 1958 por Frank Rosenblatt, que propôs o *Perceptron* [Rosenblatt 1958], um modelo de unidade com um algoritmo de aprendizagem capaz de ajustar os pesos e o viés da unidade.

1.0.5 Perceptron e Algoritmo de Aprendizagem

O Perceptron é um modelo de unidade que utiliza um algoritmo de aprendizagem supervisionada para ajustar seus pesos e viés com base em um conjunto de dados “rotulados”¹. O objetivo

¹Dados rotulados são conjuntos de dados de entrada associados a saídas desejadas, ou seja, para cada conjunto de entradas, há uma saída correta conhecida.

do Perceptron é encontrar uma função que mapeie corretamente as entradas para as saídas desejadas, minimizando o erro entre as previsões do modelo e os valores reais.

Para esclarecer o funcionamento do Perceptron e do algoritmo de aprendizagem, é proposto o Exemplo 1.1.

Exemplo 1.1 *Um investidor busca decidir qual ação adquirir com base nas variáveis: é caro?, é confiável? e é volátil?. Para isso, dispõe das avaliações de quatro colegas que também analisam essas mesmas características antes de realizar seus investimentos.*

Tabela 1.1: Banco de dados com as avaliações dos colegas e a decisão final de compra.

	valor	confiança	volatilidade	y
x_1	0	1	0	1
x_2	1	0	1	0
x_3	1	1	1	0
x_4	0	0	0	1

Em que **0** = **não**, **1** = **sim** e $x_2, \dots, x_4$ representam as respostas de cada colegas e y a decisão final de cada um. Com isso, o Perceptron terá três variáveis de entrada e como saída, a decisão sobre a compra da ação. Os pesos iniciais são definidos aleatoriamente como: $w_1 = w_2 = w_3 = 0$ e o viés como $b = 0$. A função de ativação utilizada é a **função limiar** (1.2). Testando o primeiro dado de entrada $x_1 = (0, 1, 0)$ e $y = 1$, obtém-se:

$$z = w \cdot x_1 + b = 0 \Rightarrow \hat{y} = \sigma(0) = 0$$

Como $y = 1 \neq \hat{y} = 0$, em que $\hat{y}$ é a resposta da rede definido como na Equação (1.1). O Perceptron comete um erro e é necessário aplicar o **algoritmo de aprendizagem** (a unidade de McCulloch e Pitts pararia aqui). Para isso, define-se uma função de custo, responsável por quantificar o erro cometido durante o treinamento. Neste exemplo, utiliza-se o **Erro Quadrático Médio**, dado por:

$$C(w, b) = \frac{\sum_{i=1}^k (y_i - \hat{y}_i)^2}{n}, \quad (1.5)$$

em que y_i é a resposta do dado de entrada x_i e n representa a quantidade de dados presentes no conjunto de treinamento. $C(w, b)$ não depende de w e b explicitamente, mas sim por meio de $\hat{y}$, ou seja, $C(w, b) = C(\hat{y}(w, b))$, já que $\hat{y} = \sigma(z)$ e $z = w \cdot x + b$.

O processo de treinamento ajusta iterativamente os pesos e o viés por meio do método da Descida do Gradiente (SG - *Gradient Descent*), cujo objetivo é minimizar a função de custo. Esse método utiliza as derivadas parciais da função de custo em relação aos pesos e ao viés, o que indica a direção de maior crescimento da função. Para reduzir o erro, atualizam-se os parâmetros na direção oposta ao gradiente. As derivadas são:

$$\frac{\partial C(w, b)}{\partial w_i} = -2 \cdot \frac{\sum_{i=1}^k (y_i - \hat{y}_i) \cdot \sigma'(z) \cdot x_i}{n}, \quad (1.6)$$

$$\frac{\partial C(w, b)}{\partial b_i} = -2 \cdot \frac{\sum_{i=1}^k (y_i - \hat{y}_i) \cdot \sigma'(z)}{n}. \quad (1.7)$$

No o Exemplo 1.1 assume-se $\sigma'(z) = 1$ para simplificação. Como a derivada aponta para a região de maior valor da função, multiplica-se por -1 para obter a direção da região de minimização. Com isso, é obtido o Algoritmo 1:

Entrada: Taxa de aprendizado η , função de custo $C(w, b)$, pesos iniciais

$w_1, w_2, \dots, w_n$ e viés b

Saída: Pesos e viés ajustados

Inicializar os pesos w_i e o viés b com valores pequenos aleatórios;

Repita

Calcular a saída do modelo;

$\hat{y}_i \leftarrow \sigma(\sum_{i=1}^n w_i x_i + b)$;

Calcular o erro;

$\Delta_w \leftarrow \frac{\sum_{i=1}^n (y_i - \hat{y}_i) \cdot \sigma'(z) \cdot x_i}{n}$;

$\Delta_b \leftarrow \frac{\sum_{i=1}^n (y_i - \hat{y}_i) \cdot \sigma'(z)}{n}$;

Atualizar os pesos e o viés;

$w_i \leftarrow w_i + \eta \Delta_w$, para todo $i = 1, 2, \dots, n$;

$b \leftarrow b + \eta \Delta_b$;

até convergência ou número máximo de iterações;

Retorne w_i, b ;

Algoritmo 1: Método da Descida do Gradiente.

A taxa de aprendizagem (η) presente no Algoritmo 1 pode ser interpretada como o tamanho do passo dado em direção ao mínimo da função de custo. Quando o valor dessa taxa é muito pequeno, o processo de convergência torna-se mais lento, exigindo um número maior de iterações para atingir o mínimo. Por outro lado, se o passo for muito grande, o algoritmo pode ultrapassar o ponto de mínimo, dificultando ou até impedindo a convergência adequada, para o Exemplo 1.1 é utilizada uma taxa de aprendizado de 0,1.

Aplicando o algoritmo 1 no Exemplo 1.1, os seguintes ajustes são feitos nos pesos e no viés após uma passada pelos 4 dados presentes no banco de dados:

$$w = (-0,1, 0,1, -0,1), \quad b = 0,1$$

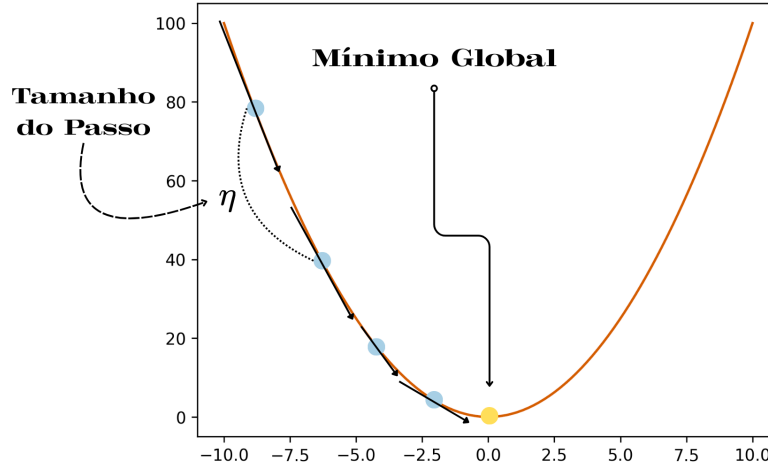
Verificando as previsões da rede para os parêmentros atualizando, é obtido:

$$x_1: (0,1,0) \Rightarrow z = 0 \cdot (-0,1) + 1 \cdot 0,1 + 0 \cdot (-0,1) + 0,1 = 0,2 \Rightarrow \hat{y} = 1;$$

$$x_2: (1,0,1) \Rightarrow z = -0,1 + 0 + -0,1 + 0,1 = -0,1 \Rightarrow \hat{y} = 0;$$

$$x_3: (1,1,1) \Rightarrow z = -0,1 + 0,1 - 0,1 + 0,1 = 0 \Rightarrow \hat{y} = 0;$$

Figura 1.4: Representação da taxa de aprendizagem.



Fonte: Elaboração do Autor.

$$x_4: (0, 0, 0) \Rightarrow z = 0 + 0 + 0 + 0, 1 = 0, 1 \Rightarrow \hat{y} = 1.$$

Basta observar que $\hat{y} = \sigma(z)$ e $\sigma(z)$ é a função de ativação utilizada. Assim, após uma única época com $\eta = 0,1$ os pesos e o viés encontrados classificam corretamente todos os exemplos do conjunto de treinamento.

Comumente o banco de dados é dividido em dois conjuntos: o conjunto de treinamento e o conjunto de teste. O conjunto de treinamento é utilizado para ajustar os pesos e o viés do Perceptron, enquanto o conjunto de teste é utilizado para avaliar o desempenho do modelo em dados não vistos durante o treinamento. Essa divisão é essencial para garantir que o modelo aprenda a generalizar bem para novos dados, evitando o problema do *overfitting* (na seção 1.2), que ocorre quando o modelo se ajusta excessivamente aos dados de treinamento, perdendo a capacidade de generalização.

1.1 Redes Neurais

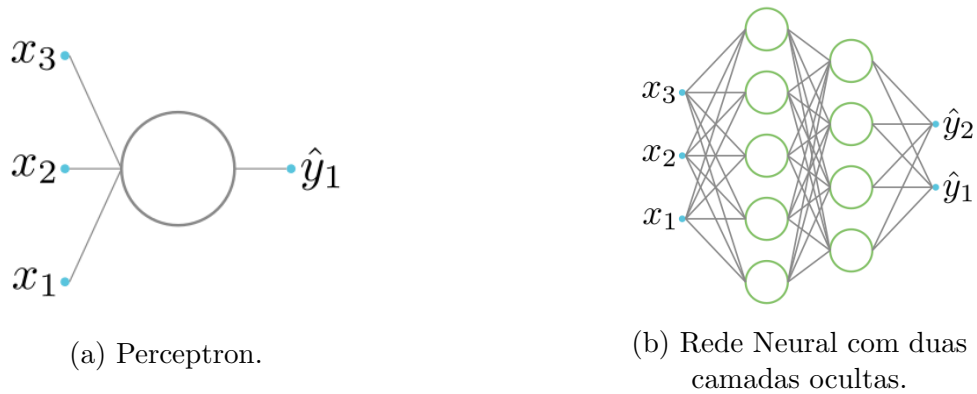
As redes neurais são um “encadeamento” de múltiplas unidades, onde a saída de uma unidade é a entrada de outra. Redes neurais podem ser organizadas em camadas: uma *camada de entrada*, uma ou mais *camadas ocultas* e uma *camada de saída*. Cada camada é composta por várias unidades e cada unidade em uma camada está conectada a todas as unidades na camada seguinte. A Figura 1.5a ilustra a estrutura básica de uma unidade vista na Seção 1.0.3. Já a Figura 1.5b mostra uma rede neural com duas camadas ocultas, em que a camada de entrada tem três unidades, a primeira camada oculta tem cinco unidades, a segunda camada oculta tem quatro unidades e a camada de saída tem duas unidades. Matematicamente, é possível representar uma rede neural com duas camadas ocultas da seguinte maneira:

$$\hat{y} = \sigma^s \left[\sum_{i=1}^4 \sigma_i^2 \left(\sum_{j=1}^5 \sigma_j^1 \left(\sum_{k=1}^3 w_{jk}^1 x_k + b_j^1 \right) w_{ij}^2 + b_i^2 \right) w_i^s + b_i^s \right], \quad (1.8)$$

em que σ_j^1 é a função de ativação da j -ésima unidade da primeira camada oculta, σ_i^2 é a função de ativação da i -ésima unidade da segunda camada oculta e σ^s é a função de ativação da s -ésima unidade da camada de saída. Os pesos e os vieses são representados por w_{jk}^1 , b_j^1 , w_{ij}^2 , b_i^2 , w_i^s e b^s . Generalizando a notação, para uma rede neural com L camadas ocultas, a saída da rede pode ser expressa como:

$$\hat{y} = \sigma^s \left[\sum_{i_L=1}^{n_L} \sigma_{i_L}^L \left(\sum_{i_{L-1}=1}^{n_{L-1}} \sigma_{i_{L-1}}^{L-1} \left(\cdots \sum_{k=1}^{n_1} \sigma_k^1 \left(\sum_{m=1}^{n_0} w_{km}^1 x_m + b_k^1 \right) w_{jk}^2 + b_j^2 \right) w_{ij}^L + b_i^L \right) w_i^s + b^s \right]. \quad (1.9)$$

Figura 1.5: Estrutura visual básica de uma unidade e uma rede neural com duas camadas ocultas.



Fonte: Elaboração do Autor.

Outra forma de representar uma rede neural é utilizando a notação matricial, em que as entradas, os pesos e os vieses são representados por matrizes e vetores. Para uma rede neural com n entradas, uma camada oculta com m neurônios e uma saída, a saída da rede pode ser expressa como:

$$\underbrace{\begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}}_{m \times n} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{n \times 1} + \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}}_{m \times 1}$$

Essa forma representa a saída de uma camada caso a rede tenha apenas uma camada oculta, para redes neurais com mais camadas ocultas, a notação matricial torna-se mais complexa, mas o princípio de multiplicação de matrizes e adição de vetores permanece o mesmo.

1.1.1 Arquiteturas de Redes Neurais

Existem diversas arquiteturas de redes neurais, cada uma projetada para lidar com diferentes tipos de problemas e dados. Algumas das arquiteturas mais comuns incluem:

- *Redes Neurais Feedforward (FNNs)*: São as redes neurais mais simples, onde a informação flui em uma única direção, da camada de entrada para a camada de saída, passando pelas camadas ocultas. FNNs são amplamente utilizadas para tarefas de classificação e regressão.
- *Redes Neurais Convolucionais (CNNs)*: São projetadas para processar dados com uma estrutura de grade, como imagens. Estas utilizam operações de convolução para extrair características espaciais dos dados de entrada. CNNs são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos e detecção de faces.
- *Redes Neurais Recorrentes (RNNs)*: São projetadas para lidar com dados sequenciais, onde a saída em um determinado momento depende das entradas anteriores. RNNs possuem conexões recorrentes que permitem que informações sejam mantidas ao longo do tempo. Elas são amplamente utilizadas em tarefas como processamento de linguagem natural e reconhecimento de fala.
- *Redes Neurais Generativas Adversariais (GANs)*: Consistem em duas redes neurais que competem entre si: um gerador e um discriminador. O gerador cria dados falsos, enquanto o discriminador tenta distinguir entre dados reais e falsos. GANs são amplamente utilizadas para geração de imagens realistas e outras tarefas criativas.
- *Encoder-Decoder*: São arquiteturas que consistem em duas partes principais: o encoder, que comprime os dados de entrada em uma representação latente, e o decoder, que reconstrói os dados a partir dessa representação. Essas redes são amplamente utilizadas em tarefas como tradução automática e geração de texto.

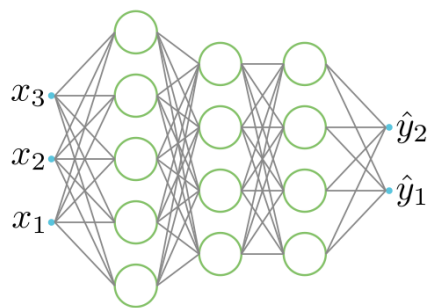
É possível utilizar duas arquiteturas em conjunto, como abordado na seção 1.4.

As Figuras 1.6a e 1.6c ilustram a arquitetura de uma Rede Neural Feedforward, com o funcionamento semelhante ao desenvolvido na Equação (1.9). A Figura 1.6b mostra a arquitetura de uma Rede Neural Recorrente, estas redes possuem conexões recorrentes, ou seja, a saída de uma unidade é realimentada como entrada para a mesma unidade ou para unidades anteriores, permitindo que informações sejam mantidas ao longo do tempo, por fungir do escopo deste trabalho, não serão abordados os detalhes matemáticos de funcionamento das RNNs.

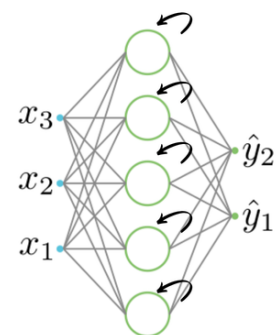
1.1.2 Backpropagation

Assim como no perceptron, as redes neurais utilizam algoritmos de aprendizagem para ajustar seus pesos e vieses com o objetivo de minimizar uma função de custo. O algoritmo mais comum utilizado para esse propósito é o *backpropagation* (retropropagação) [Hecht-Nielsen 1992] e [Werbos 1990], que é uma extensão do método da descida do gradiente. O backpropagation é utilizado para calcular os gradientes da função de custo em relação aos pesos e vieses de todas as camadas da rede neural, permitindo a atualização eficiente desses parâmetros durante o treinamento. O algoritmo funciona propagando o erro da saída da rede de volta para as camadas anteriores, ajustando os pesos com base na contribuição de cada unidade para o erro total.

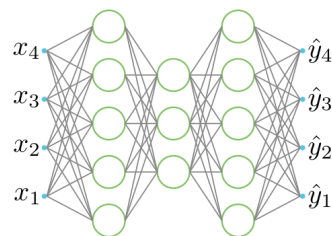
Figura 1.6: Arquiteturas de Redes Neurais.



(a) Redes Neurais Feedforward (FNNs).



(b) Redes Neurais Recorrentes (RNNs).



(c) Encoder-Decoder.

Fonte: Elaboração do Autor.

Esse processo é repetido iterativamente até que a função de custo seja minimizada, resultando em uma rede neural treinada capaz de realizar previsões precisas em novos dados. Essa propagação do erro é feita utilizando a regra da cadeia do cálculo diferencial, que permite calcular as derivadas parciais de funções compostas. O algoritmo de backpropagation pode ser resumido nos seguintes passos, apresentados no Algoritmo 2.

Entrada: Dados de treinamento (x, y) , taxa de aprendizado η , pesos W e vieses b

Saída: Pesos W e vieses b atualizados

Repita

Forward Pass: propagar x pela rede (camada a camada) e obter $\hat{y}$;

Cálculo do Erro: calcular o erro comparando $\hat{y}$ com y usando uma função de custo ;

Backward Pass: retropropagar o erro e calcular os gradientes $\nabla_W C$ e $\nabla_b C$ (regra da cadeia);

Atualização: atualizar pesos e vieses (descida do gradiente);

$W \leftarrow W - \eta \nabla_W C$;

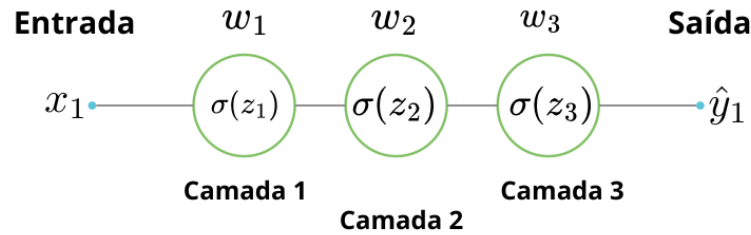
$b \leftarrow b - \eta \nabla_b C$;

até critério de parada (*ex.: n^o de épocas ou convergência*);

Algoritmo 2: Etapas do treinamento via Backpropagation.

A Figura 1.7 ilustra mostra uma rede neural simples e a equação 1.10 mostra o cálculo do gradiente para ajustar o peso da primeira camada (w_1) utilizando a regra da cadeia,

Figura 1.7: Ilustração do processo de backpropagation em uma rede neural.



Fonte: Elaboração do Autor,

$$\nabla_w C = \frac{\partial C(w, b)}{\partial w_1} = \frac{C(w, b)}{\sigma(z_3)} \cdot \frac{\sigma(z_3)}{z_3} \cdot \frac{z_3}{\sigma(z_2)} \cdot \frac{\sigma(z_2)}{z_2} \cdot \frac{z_2}{\sigma(z_1)} \cdot \frac{\sigma(z_1)}{z_1} \cdot \frac{z_1}{w_1}, \quad (1.10)$$

em que $z_1 = w_1 \cdot x + b_1$, $z_2 = \sigma(z_1) \cdot w_2 + b_2$ e $z_3 = \sigma(z_2) \cdot w_3 + b_3$ são as saídas das camadas ocultas e $C(w, b)$ é a função de custo do erro quadrático médio, ou seja, $C(w, b) = \frac{(\hat{y} - \sigma(z_3))^2}{n}$, n

sendo o número de amostras no conjunto de dados.

1.1.3 Gradiente Descendente Estocástico

O Gradiente Descendente Estocástico (SGD - Stochastic Gradient Descent) [Ruder 2016] é uma variação do método de descida do gradiente tradicional, amplamente utilizado no treinamento de redes neurais e outros modelos de aprendizado de máquina. A principal diferença entre o SGD e o método de descida do gradiente padrão é a forma como os gradientes são calculados e atualizados durante o treinamento. No método de descida do gradiente tradicional, os gradientes são calculados usando todo o conjunto de dados de treinamento, ou seja, os pesos são atualizados uma vez em cada época de treinamento, o que pode ser computacionalmente caro e demorado, especialmente para grandes conjuntos de dados. Em contraste, o SGD calcula os gradientes usando apenas uma amostra aleatória (ou um pequeno lote) do conjunto de dados em cada iteração, ou seja, caso o meu conjunto de dados tenha uma tamanho de 1000 e o tamanho do lote seja 10, o SGD atualizará os pesos 100 vezes por época. Isso torna o processo de atualização dos pesos mais rápido e eficiente, permitindo que o modelo aprenda mais rapidamente.

A Figura 1.8a ilustra como funciona o Gradiente Descendente Estocástico, em que cada seta preta indica uma atualização dos pesos do modelo. A Figura 1.8b ilustra o caminho percorrido pelo método de descida do gradiente tradicional, enquanto a Figura 1.8c ilustra o caminho percorrido pelo SGD.

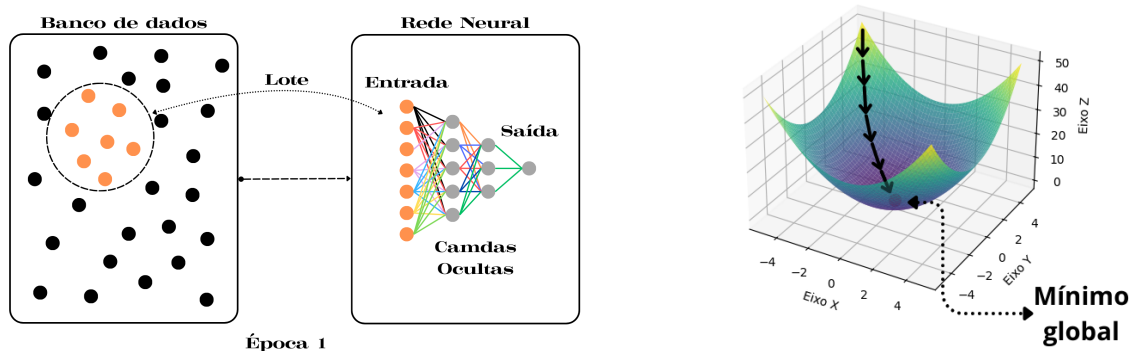
Apesar da Descida do Gradiente padrão fornecer uma direção mais precisa para o mínimo global da função de custo, o SGD é capaz de chegar próximo do mínimo de forma mais rápida e eficiente.

1.2 Overfitting

O *Overfitting* [Deep Learning Book 2025] (do inglês “Sobreajuste” ou “Ajuste Excessivo”) é um problema comum em aprendizado de máquina, onde um modelo aprende os detalhes e o ruído dos dados de treinamento a tal ponto que ele se torna incapaz de generalizar para novos dados. Isso significa que o modelo tem um desempenho excelente nos dados de treinamento, mas falha ao fazer previsões precisas em dados não vistos anteriormente.

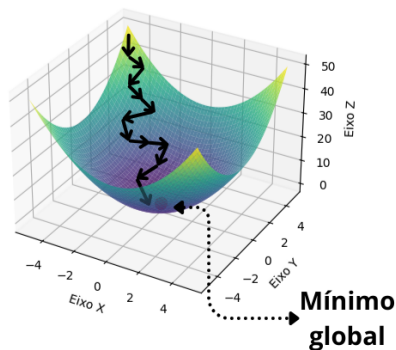
O overfitting ocorre quando o modelo é muito complexo em relação à quantidade e qualidade dos dados disponíveis para treinamento. Modelos com muitos parâmetros, como redes neurais profundas, são particularmente suscetíveis ao overfitting, especialmente quando o conjunto de dados de treinamento é pequeno ou contém ruído significativo. Na figura 1.9 é apresentada uma ilustração do overfitting, em que os pontos vermelhos são os dados de treinamento e os pontos azuis são os dados de teste, a curva verde representa um modelo que se ajusta perfeitamente aos dados de treinamento, mas falha em generalizar para novos dados, enquanto a curva azul seria um modelo ideal mais general.

Figura 1.8: Comparação entre o método de descida do gradiente tradicional e o Gradiente Descendente Estocástico.



(a) Separando em Lotes o banco de dados para o SGD.

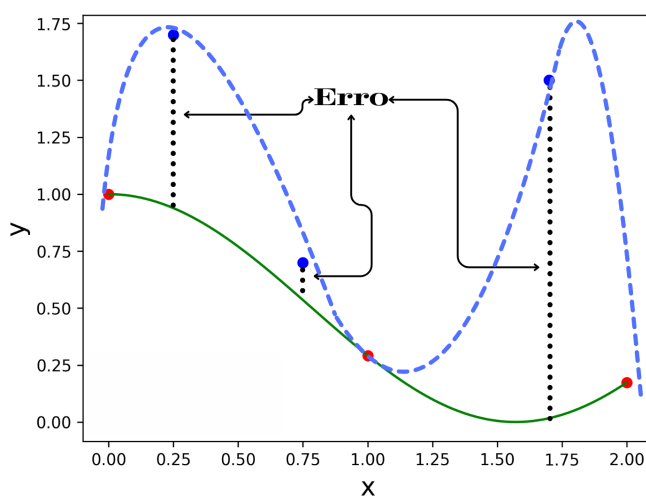
(b) Descida do Gradiente Tradicional.



(c) Gradiente Descendente Estocástico (SGD).

Fonte: Elaboração do Autor.

Figura 1.9: Ilustração do Overfitting.



Fonte: Elaboração do Autor.

1.2.1 Hiperparâmetros

Um algoritmo de aprendizado pode ser visto como uma função que recebe dados de treinamento como entrada e produz como saída uma função ou modelo (i.e um conjunto de funções). No entanto, na prática muitos algoritmos de aprendizado possuem parâmetros adicionais que não são aprendidos diretamente a partir dos dados de treinamento, mas que influenciam o processo de aprendizado. Esses parâmetros são chamados de *hiperparâmetros* [Bengio 2012] e sua escolha adequada é crucial para o desempenho do modelo. Alguns exemplos comuns de hiperparâmetros incluem:

- *Taxa de Aprendizado*: Controla o tamanho dos passos dados durante a atualização dos pesos do modelo. Uma taxa de aprendizado muito alta pode levar a oscilações e impedir a convergência, enquanto uma taxa muito baixa pode resultar em um processo de treinamento lento. Valores típicos para a taxa de aprendizado variam entre 0,01 e 0,1, dependendo do problema e do algoritmo utilizado.
- *Número de Épocas*: Define quantas vezes o algoritmo de aprendizado passará por todo o conjunto de dados de treinamento. Um número muito alto pode levar ao overfitting, enquanto um número muito baixo pode resultar em underfitting (subajuste). Uma técnica comum para determinar o número ideal de épocas é o uso do *Early Stopping* (do inglês Parada Antecipada), que monitora o desempenho do modelo em um conjunto de validação e interrompe o treinamento quando o desempenho começa a piorar.
- *Tamanho do Lote (Batch Size)*: Refere-se ao número de amostras de dados usadas para calcular o gradiente em cada iteração do treinamento. Tamanhos de lote menores podem levar a atualizações mais ruidosas, enquanto tamanhos maiores podem resultar em um treinamento mais estável, embora mais lento. Os valores escolhidos geralmente variam entre 1 e 32, sendo que valores acima de 10 tiram proveito da Unidade Central de Processamento (do inglês *Central Processing Unit* - CPU) e da Unidade de Processamento Gráfico (do inglês *Graphics Processing Unit* - GPU), que oferece ganho de velocidade dos produtos matriz-matriz em relação aos produtos matriz-vetor.
- *Arquitetura do Modelo*: Inclui o número de camadas, o número de unidades em cada camada e o tipo de funções de ativação utilizadas. Modelos mais complexos podem capturar padrões mais sofisticados, mas também são mais propensos ao overfitting.

1.2.2 Regularização

A regularização é uma técnica utilizada para prevenir o overfitting em modelos de aprendizado de máquina, adicionando uma penalidade à função de custo que o modelo tenta minimizar durante o treinamento. Essa penalidade incentiva o modelo a manter os pesos dos parâmetros pequenos, o que ajuda a evitar que ele se ajuste excessivamente aos dados de treinamento. Existem várias técnicas de regularização, sendo as mais comuns:

- *Regularização L1 (Norma 1)*: Adiciona uma penalidade proporcional à soma dos valores absolutos dos pesos dos parâmetros à função de custo. Isso pode levar a soluções esparsas, onde alguns pesos são reduzidos a zero, efetivamente removendo certas características do modelo. A equação da função de custo com regularização L1 é dada por:

$$C(w, b) = C_0(w, b) + \frac{\lambda}{n} \sum_i |w_i|, \quad (1.11)$$

em que $C_0(w, b)$ é a função de custo original, λ é o parâmetro de regularização que controla a força da penalidade, w_i são os pesos dos parâmetros do modelo e n é o número de amostras no conjunto de dados.

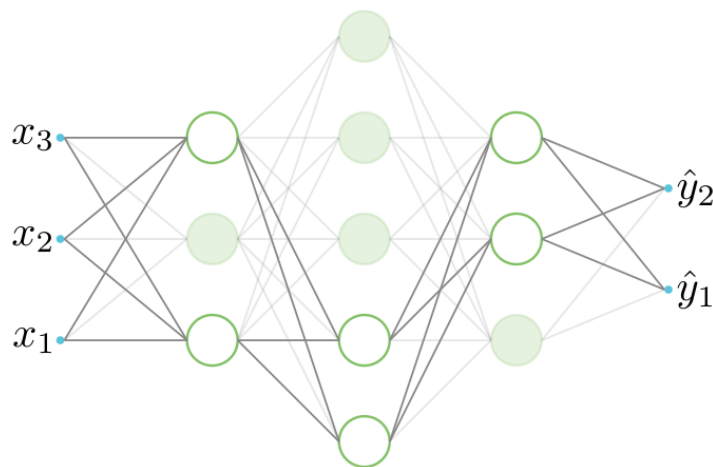
- *Regularização L2 (Norma 2)*: Adiciona uma penalidade proporcional à soma dos quadrados dos pesos dos parâmetros à função de custo. Isso incentiva o modelo a distribuir os pesos de forma mais uniforme, evitando que alguns pesos se tornem muito grandes. A equação da função de custo com regularização L2 é dada por:

$$C(w, b) = C_0(w, b) + \frac{\lambda}{n} \sum_i w_i^2, \quad (1.12)$$

em que os termos são definidos da mesma forma que na regularização L1.

- *Dropout*: Durante o treinamento, essa técnica desativa aleatoriamente uma fração das unidades em cada camada da rede neural para cada época. Isso força a rede a aprender representações mais robustas e reduz a dependência de unidades específicas, ajudando a prevenir o overfitting. Em termos gerais, ao aplicar o dropout estamos treinando uma coleção de sub-redes dentro da rede original, o que pode promover a generalização do modelo.

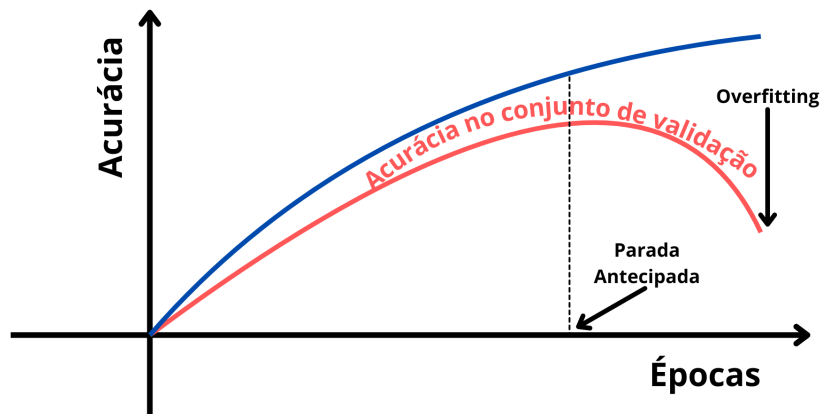
Figura 1.10: Ilustração do Dropout removendo 60% das unidades aleatoriamente durante uma época de treinamento.



Fonte: Elaboração do Autor.

- *Early Stopping* (Parada Antecipada): Monitora o desempenho do modelo em um conjunto de validação durante o treinamento e interrompe o processo quando o desempenho começa a piorar. Isso evita que o modelo continue a se ajustar aos dados de treinamento além do ponto ideal. Como ilustrado na Figura 1.11, o ponto de parada ideal é onde a acurácia na validação começa a diminuir, indicando o início do overfitting.

Figura 1.11: Ilustração do Early Stopping.



Fonte: Elaboração do Autor.

1.2.3 Momentum

O *Momentum* (Momento) [Sutskever 2013] é uma técnica utilizada para acelerar o processo de treinamento de redes neurais, especialmente em situações onde a função de custo apresenta regiões íngremes ou planas. A ideia principal do momentum é acumular uma média ponderada dos gradientes anteriores para suavizar as atualizações dos pesos, permitindo que o modelo mantenha uma direção consistente durante o treinamento. Isso ajuda a evitar oscilações e acelera a convergência para o mínimo da função de custo. Dada uma função de custo $C(w, b)$ a ser minimizada, o momentum clássico é definido por:

$$\begin{cases} v_{t+1} = \beta v_t - 1 + \epsilon \nabla C(w_t, b_t) \\ \theta_{t+1} = \theta_t + v_{t+1}, \end{cases} \quad (1.13)$$

em que $\epsilon > 0$ é a taxa de aprendizagem, $\beta \in [0, 1]$ é o coeficiente de momentum, v_t é a velocidade acumulada no tempo t , $\theta_t = (w_t, b_t)$ representa os parâmetros do modelo (pesos e vieses) no tempo t e $\nabla C(w_t, b_t)$ é o gradiente da função de custo em relação aos parâmetros do modelo. O coeficiente de momentum β controla a influência dos gradientes anteriores nas atualizações atuais. Valores típicos para β variam entre 0,9 e 0,99, com valores mais altos dando mais peso aos gradientes passados.

1.2.4 Vanishing Gradient

O problema do *vanishing gradient* (gradiente que desaparece) ocorre durante o treinamento de redes neurais profundas, onde os gradientes calculados durante o processo de backpropagation tornam-se extremamente pequenos à medida que são propagados para as camadas mais próximas da entrada. Isso acontece devido à multiplicação repetida de pequenas derivadas associadas às funções de ativação e aos pesos das camadas. Como resultado, os pesos das camadas iniciais da rede recebem atualizações insignificantes, dificultando o aprendizado efetivo dessas camadas.

Exemplo 1.2 Considere uma rede neural profunda com várias camadas ocultas, onde cada camada utiliza a função de ativação sigmoid logística, definida como:

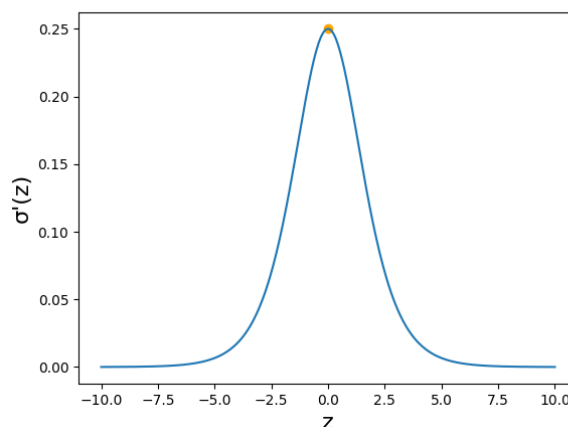
$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.14)$$

A derivada dessa função é dada por:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (1.15)$$

Durante o processo de backpropagation, o gradiente da função de custo em relação aos pesos das camadas iniciais é calculado utilizando a regra da cadeia. Isso envolve a multiplicação das derivadas das funções de ativação de cada camada, como mostra a Equação 1.10. Como a derivada da função sigmoid é sempre menor que 0,25 (o valor máximo ocorre em $z = 0$), a multiplicação repetida dessas pequenas derivadas resulta em gradientes que se tornam cada vez menores à medida que são propagados para trás através das camadas da rede. Por exemplo, se uma rede possui 10 camadas ocultas, o gradiente pode ser reduzido a um valor extremamente pequeno, como 10^{-6} ou menor, tornando as atualizações dos pesos nas camadas iniciais praticamente insignificantes.

Figura 1.12: Gráfico da derivada da função sigmoid logística.



Fonte: Elaboração do Autor.

O problema do vanishing gradient é particularmente comum em redes neurais recorrentes (RNNs) e em redes feedforward profundas que utilizam funções de ativação como a sigmoid ou a tangente hiperbólica. Para contornar o problema do vanishing gradient, várias técnicas podem ser empregadas, incluindo:

- *Uso de Funções de Ativação Alternativas:* Funções de ativação como ReLU (Rectified Linear Unit) e suas variantes (Leaky ReLU, Parametric ReLU) têm derivadas constantes para valores positivos, o que ajuda a manter os gradientes maiores durante o backpropagation.
- *Batch Normalization:* Normaliza as ativações de cada camada durante o treinamento, o que pode ajudar a estabilizar os gradientes e acelerar o processo de aprendizado.

1.2.5 Córtex Visual Humano e Visão Computacional

O córtex visual humano é a parte do cérebro responsável pelo processamento das informações visuais recebidas pelos olhos [Stanfield 2014]. Ele está localizado na parte posterior do cérebro, na região occipital. O córtex visual é organizado em várias áreas, cada uma especializada em processar diferentes aspectos da visão, como cor, forma, movimento e profundidade. Essas áreas trabalham em conjunto para interpretar e compreender o mundo visual ao nosso redor. A estrutura do córtex visual é altamente complexa, com bilhões de unidades interconectadas que formam redes neurais intrincadas. Essas redes são capazes de aprender e adaptar-se com base nas experiências visuais, permitindo-nos reconhecer objetos, rostos e padrões visuais de maneira eficiente. Uma característica importante do córtex visual é a presença de campos receptivos, que são regiões específicas do campo visual onde a estimulação de uma unidade resulta em uma resposta. Esses campos receptivos são organizados de forma hierárquica, com unidades em níveis mais baixos respondendo a características simples, como bordas e texturas, enquanto unidades em níveis mais altos respondem a características mais complexas, como formas e objetos completos. É por meio desse mecanismo e também pela capacidade de aprendizagem que o ser humano é capaz de reconhecer e interpretar o mundo visual ao seu redor.

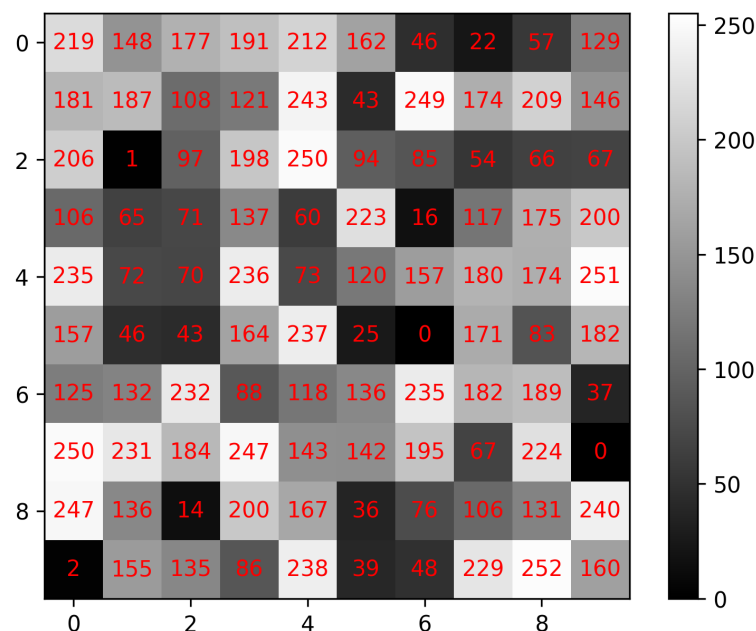
Traduzir a visão para uma representação digital é um processo extremamente complexo. Em parte, porque é um problema inverso, no qual busca-se recuperar algumas variáveis desconhecidas a partir de informações insuficientes para especificar completamente a solução. Portanto, deve-se recorrer a modelos baseados na física, modelos probabilísticos ou aprendizado de máquina com grandes conjuntos de exemplos para resolver ambiguidades entre possíveis soluções. Entretanto, modelar o mundo visual em toda a sua rica complexidade é muito mais difícil do que, por exemplo, modelar o trato vocal que produz sons da fala [Szeliski 2010].

É surpreendente que humanos e animais façam isso com tanta facilidade, enquanto algoritmos de visão computacional são tão propensos a erros. Pessoas que nunca trabalharam na área frequentemente subestimam a dificuldade do problema. Essa percepção equivocada de que a visão deveria ser fácil remonta aos primeiros dias da inteligência artificial, quando se

acreditava inicialmente que as partes cognitivas (prova lógica e planejamento) da inteligência eram intrinsecamente mais difíceis do que os componentes perceptivos [Müller 2008]².

Uma imagem virtual é uma representação visual de um objeto, cena ou conceito, composta por uma matriz de pixels que contém informações sobre cor e intensidade. Cada pixel é uma célula na matriz da imagem que possui um valor específico. As imagens podem ser em escala de cinza, onde cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco), como mostra a Figura 1.13, ou em cores, onde cada pixel é representado por uma combinação de valores para as cores primárias (vermelho, verde e azul - RGB), uma imagem colorida pode ser interpretada como a sobreposição de três matrizes, cada matriz representando uma das cores primárias.

Figura 1.13: Exemplo de uma imagem/matriz 10×10 em escala de cinza, onde cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco).



Fonte: Elaboração do Autor.

1.2.6 Dígitos Manuscritos

Para exemplificar o funcionamento de como uma rede neural faz o reconhecimento de imagens, considere a tarefa de classificação de dígitos manuscritos utilizando o conjunto de dados MNIST [Yann LeCun 1998]. O conjunto de dados MNIST consiste em 70.000 imagens em escala de cinza de dígitos manuscritos (0 a 9), cada uma com uma resolução de 28×28 pixels, como mostra a Figura 1.14.

Cada imagem é representada como uma matriz de 28×28 , onde cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco). Para compor a camada de entrada da rede

²[Müller 2008] cita [Crevier 1993] como a fonte original. Entretanto o artigo original foi escrito por Seymour Papert (1966) e envolveu toda uma turma de estudantes.

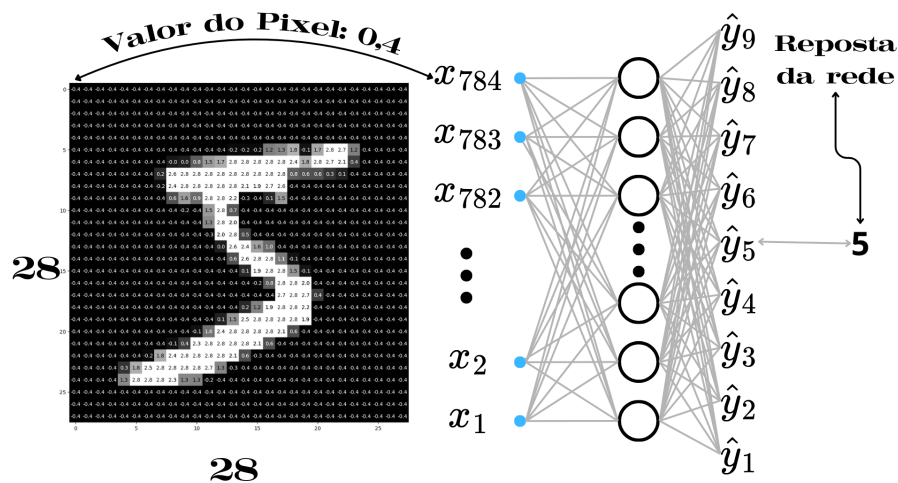
Figura 1.14: Exemplos de dígitos manuscritos do conjunto de dados MNIST.



Fonte: Elaboração do Autor.

neural, cada imagem é “achatada” em um vetor de 784 elementos ($28 \times 28 = 784$), onde cada elemento corresponde à intensidade de um pixel. O objetivo da rede neural feedforward com camadas totalmente conectadas é aprender a classificar corretamente cada imagem em uma das 10 classes correspondentes aos dígitos de 0 a 9. Como a camada de saída da rede neural deve ter 10 unidades (um para cada classe), a arquitetura da rede pode ser definida como [784, 30, 9], onde a camada de entrada tem 784 unidades, a camada oculta tem 30 unidades (valor escolhido aleatoriamente) e a camada de saída tem 9 unidades, como mostra a Figura 1.15.

Figura 1.15: Arquitetura da rede neural para classificação de dígitos manuscritos. É enviado o número de pixels da imagem (784) para a camada de entrada (unidade $x_{784} = 0,4$ intensidade do pixel), que é processada por uma camada oculta com 30 unidades, e a camada de saída tem 9 unidades, correspondendo às 10 classes de dígitos (0 a 9).



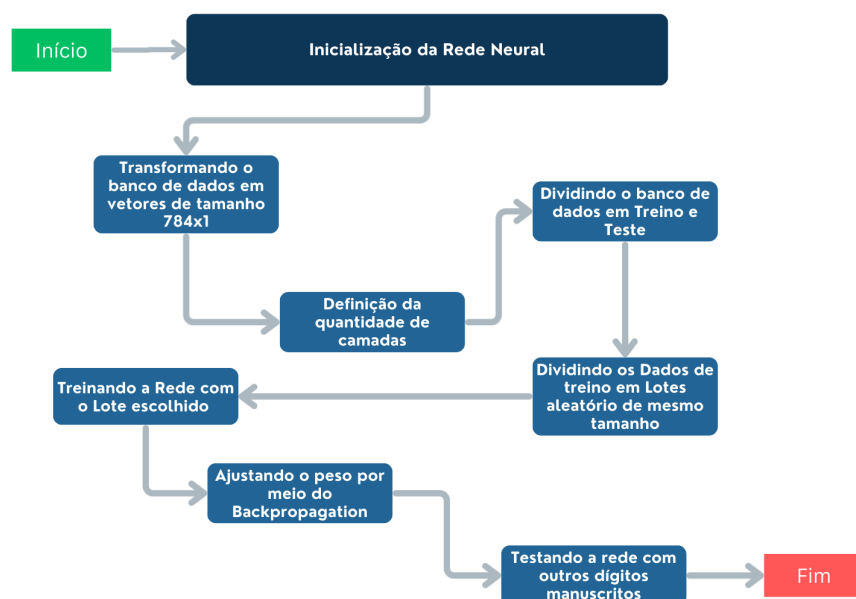
Fonte: Elaboração do Autor.

A função de ativação utilizada é a função sigmoid logística vista na Equação 1.3, que é

comumente usada em tarefas de classificação. A saída da rede é um vetor de 10 elementos, onde cada elemento representa a probabilidade de a imagem pertencer a uma das classes de dígitos. A classe com a maior probabilidade é escolhida como a previsão da rede para a imagem de entrada (para a Figura 1.15 a resposta da rede é 5, que foi a unidade com a maior probabilidade). Outra forma de interpretar a saída da rede é utilizando a Função Limiar (Equação 1.2) e quatro neurônios na camada de saída, em que cada neurônio representaria um valor binário (0 ou 1), então caso a rede recebesse o dígito 9 e sua resposta fosse correta, a saída seria 1001 que é nove em binário. O problema de treinar uma rede neural com essa arquitetura é que ela estaria mais suscetível a erro, pois errando qualquer uma das quatro saídas, a resposta seria incorreta, enquanto utilizando a função sigmoid logística, mesmo que a rede errasse a previsão da classe correta, ela poderia ainda assim atribuir uma probabilidade alta para a classe correta, o que é mais tolerante a erros.

Para o treinamento da rede é utilizado o algoritmo de backpropagation (Algoritmo 2) e o Gradiente Descendente Estocástico, com uma taxa de aprendizagem de 0,1 e a normalização $L1$ visto na Equação 1.11. Treinaremos a rede ao longo de 30 épocas. O banco de dados é dividido em um conjunto de treinamento com 60.000 imagens e um conjunto de teste com 10.000 imagens. Após o treinamento, a rede neural é avaliada no conjunto de teste para medir sua acurácia, que é a proporção de previsões corretas em relação ao total de previsões feitas. Com as configurações mencionadas, a rede neural alcançou uma acurácia de aproximadamente 95% no conjunto de teste, o que indica que ela foi capaz de aprender a classificar corretamente a maioria das imagens de dígitos manuscritos. A Figura 1.16 mostra o fluxo de dados durante o processo de treinamento e avaliação da rede neural para classificação de dígitos manuscritos.

Figura 1.16: Fluxo de dados durante o processo de treinamento e avaliação da rede neural.



Fonte: Elaboração do Autor.

1.3 Redes Neurais Convolucionais

As redes neurais convolucionais (RNCs) são uma classe especial de redes neurais projetadas para processar dados com uma estrutura de grade e que tenham uma relação de vizinhança entre os elementos, como imagens [Vinícius Trevisan 2021], foram extremamente baseadas do Cotex Visual Humano 1.2.5. Estas são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos, detecção de faces e segmentação de imagens. As RNCs são inspiradas na organização do córtex visual dos animais, onde diferentes regiões do cérebro são responsáveis por processar diferentes partes do campo visual.

1.3.1 Convolução

A operação de convolução é uma operação matemática linear entre duas funções (assim como soma e multiplicação de funções). Sejam $x(t)$ e $w(a)$ funções reais contínuas e integráveis, respectivamente denominadas de *senal* e *kernel*, em que t pode ser representado pela variável **tempo**, a convolução entre essas duas funções é definida como:

$$s(t) = (x * w)(t) = \int_{-\infty}^{\infty} x(a)w(t-a)da, \quad (1.16)$$

em que $*$ denota a operação de convolução.

Exemplo 1.3 *Seja $f(t) = e^{-t}u(t)$ e $g(t) = u(t)$, onde $u(t)$ é a função degrau unitário (Equação 1.2). A convolução $(f * g)(t)$ é dada por:*

$$(f * g)(t) = \int_{-\infty}^{\infty} e^{-a}u(a)u(t-a)da.$$

*Para $t < 0$, a função degrau unitário $u(t-a)$ é zero para todos os valores de a , o que resulta em $(f * g)(t) = 0$. Para $t \geq 0$, a função degrau unitário $u(t-a)$ é igual a 1 para $a \leq t$ e zero para $a > t$. Portanto, a convolução pode ser reescrita como:*

$$(f * g)(t) = \int_0^t e^{-a}da.$$

Calculando a integral, obtemos:

$$(f * g)(t) = [-e^{-a}]_0^t = 1 - e^{-t}.$$

Portanto, a convolução entre as funções $f(t)$ e $g(t)$ é dada por:

$$(f * g)(t) = \begin{cases} 0, & t < 0 \\ 1 - e^{-t}, & t \geq 0 \end{cases} \quad (1.17)$$

Intuitivamente, a função g é invertida e deslocada por t e depois multiplicada ponto a ponto com a função f , e o resultado é integrado sobre todo o domínio. A definição de convolução

para domínios discretos é dada por:

$$s[t] = (x * w)[t] = \sum_{a=-\infty}^{\infty} x[a]w[t-a]. \quad (1.18)$$

Exemplo 1.4 *Seja $x[n] = \{1, 2, 3\}$ e $w[n] = \{0, 1\}$, em que $x[0] = 1$, $x[1] = 2$, $x[2] = 3$, $w[0] = 0$ e $w[1] = 1$ e $x[n] = 0$ para outros valores de n . A convolução $(x * w)[n]$ é dada por:*

$$s[n] = (x * w)[n] = \sum_{a=0}^4 x[a]w[n-a],$$

onde o limite superior da soma é 4 porque a convolução entre duas sequências de comprimento M e N resulta em uma sequência de comprimento $M + N - 1$. Calculando os valores para cada n , obtemos:

$$\begin{aligned} (n=0) \quad (x * w)[0] &= x[0]w[0] + x[1]w[-1] + x[2]w[-2] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 = 0 \\ (n=1) \quad (x * w)[1] &= x[0]w[1] + x[1]w[0] + x[2]w[-1] = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot 0 = 1 \\ (n=2) \quad (x * w)[2] &= x[0]w[2] + x[1]w[1] + x[2]w[0] = 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 0 = 2 \\ (n=3) \quad (x * w)[3] &= x[0]w[3] + x[1]w[2] + x[2]w[1] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 1 = 3 \\ (n=4) \quad (x * w)[4] &= x[0]w[4] + x[1]w[3] + x[2]w[2] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 = 0 \end{aligned}$$

Portanto, a convolução entre as sequências $x[n]$ e $w[n]$ é dada por:

$$(x * w)[n] = \{0, 1, 2, 3, 0\}. \quad (1.19)$$

O tamanho do vetor resultante da convolução entre dois vetores de comprimento M e N é dado por $M + N - 1$. No exemplo acima, a convolução entre os vetores $x[n]$ e $w[n]$ resultou em um vetor de comprimento 5, que é igual a $3 + 2 - 1 = 4$. Para voltarmos para o comprimento original do vetor $x[n]$, pode-se fazer um truncamento do vetor resultante, ou seja, remover a mesma quantidade de elementos do início e do final do vetor resultante, o que é conhecido como convolução “válida”. No exemplo acima, removendo o primeiro e o último elemento do vetor resultante, obtemos a convolução válida:

$$(x * w)_{\text{valid}}[n] = \{1, 2, 3\}. \quad (1.20)$$

No contexto de redes neurais convolucionais, a convolução é aplicada a dados discretos, como imagens representadas por matrizes de pixels. Nesse caso, o sinal x é uma matriz bidimensional que representa a imagem de entrada e o *kernel* w é uma matriz menor que atua como um filtro para extrair características específicas da imagem. A operação de convolução em dados discretos é realizada deslizando o *kernel* sobre a imagem e calculando a soma ponderada dos valores dos pixels cobertos pelo *kernel*. A saída da convolução é uma nova matriz, chamada de mapa de características, que destaca as características extraídas pela aplicação do *kernel*.

No caso de uma convolução bidimensional, a operação é definida como:

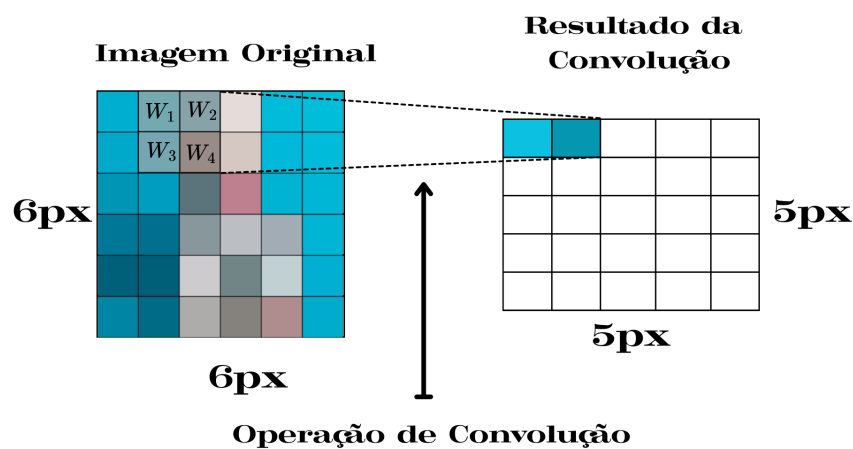
$$S(i, j) = (X * W)(i, j) = \sum_m \sum_n X(m, n)W(i - m, j - n), \quad (1.21)$$

considerando que X é a matriz da imagem de entrada, W é a matriz do kernel e S é o mapa de características resultante.

Exemplo 1.5 P

mudar imagem (feia)

Figura 1.17: Exemplo de convolução bidimensional. A função $X(t)$ é convoluída com o *kernel* $W(a)$ para produzir a saída $S(t)$.



Fonte: Elaboração do Autor.

Na Figura 1.17 é ilustrado um exemplo de convolução bidimensional, onde uma matriz de entrada X é convoluída com um *kernel* W para produzir a saída S . O *kernel* é deslizado sobre a matriz de entrada, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo *kernel* é calculada para gerar o valor correspondente na matriz de saída. O cálculo da largura e da altura da imagem de saída pode ser feito utilizando as seguintes fórmulas:

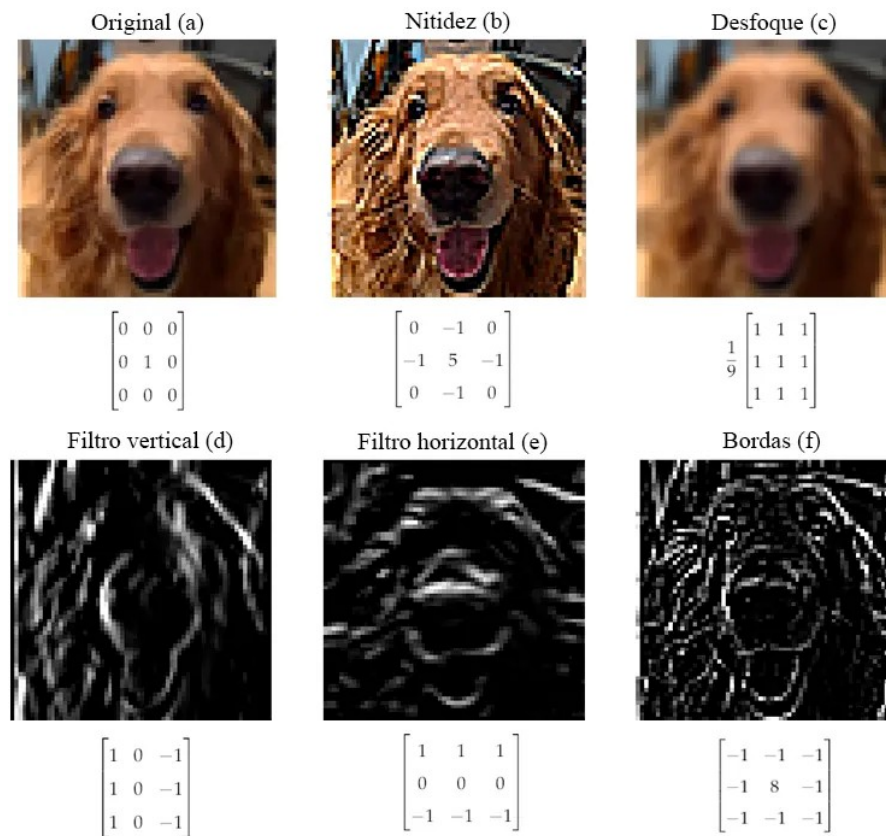
$$w_{out} = w_{in} - w_{kernel} + 1, \quad (1.22)$$

$$h_{out} = h_{in} - h_{kernel} + 1. \quad (1.23)$$

Em que w_{out} e h_{out} são a largura e a altura da imagem de saída, w_{in} e h_{in} são a largura e a altura da imagem de entrada, e w_{kernel} e h_{kernel} são a largura e a altura do *kernel*, respectivamente. A Figura 1.18 mostra diferentes aplicações de *kernels* em uma imagem.

Normalmente os *kernels* são muito menores do que as imagens, e através da operação de convolução um *kernel* é utilizado para processar toda uma imagem, portanto, diz-se que os parâmetros são compartilhados. Por conta dessa representação reduzida, cada camada da rede convolucional deve aprender uma quantidade menor de parâmetros.

Figura 1.18: Aplicações de diferentes *kernels* em uma imagem. A imagem original é representada em (a), juntamente com o *kernel* identidade, que não faz modificações. Em (b) a imagem passou por um *kernel* que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um *kernel* de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um *kernel* que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O *kernel* em (f) reforça os contornos (bordas) da imagem.



Fonte: Adaptado de [Vinícius Trevisan 2021]

Em CNNs, normalmente consideramos a convolução como uma camada da rede, e cada camada normalmente possui múltiplos *kernels*, produzindo a mesma quantidade de mapas de atributos, cada um responsável por extrair diferentes características da imagem de entrada. Durante o treinamento da rede, os valores dos *kernels* são ajustados para otimizar a capacidade da rede de reconhecer padrões específicos nas imagens. Por exemplo, uma camada convolucional com 3 *kernels* que recebe como entrada uma imagem em escala de cinza de dimensões $(w_{in} \times h_{in} \times 1)$ produzirá uma saída com dimensões $(w_{out} \times h_{out} \times 3)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em 1.22 e 1.23. A saída não é mais uma imagem comum, mas sim um conjunto de três mapas de características, cada um representando a resposta da imagem de entrada a um dos três *kernels* aplicados. Encadeando os mapas de atributos, a rede pode aprender representações hierárquicas das características presentes nas imagens, desde bordas e texturas simples até formas e objetos mais complexos.

1.3.2 Kernel Multicamadas

Em uma imagem com mais de um canal, como no caso de uma imagem RGB (vermelho, verde, azul) do inglês (*Red, Green, Blue*), a convolução acontece com um *kernel* em cada canal da imagem, um conjunto de canais é chamado de filtro. Por exemplo, considere uma imagem RGB de dimensões $(w_{in} \times h_{in} \times 3)$ e um *kernel* de dimensões $(k_w \times k_h \times 3)$. A convolução é realizada separadamente em cada canal (vermelho, verde e azul) da imagem, utilizando o respectivo canal do *kernel*. Após a aplicação do *kernel* em cada canal, os resultados são somados ou é aplicada uma média para produzir um único mapa de características. Portanto, a saída da convolução terá dimensões $(w_{out} \times h_{out} \times 1)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em 1.22 e 1.23.

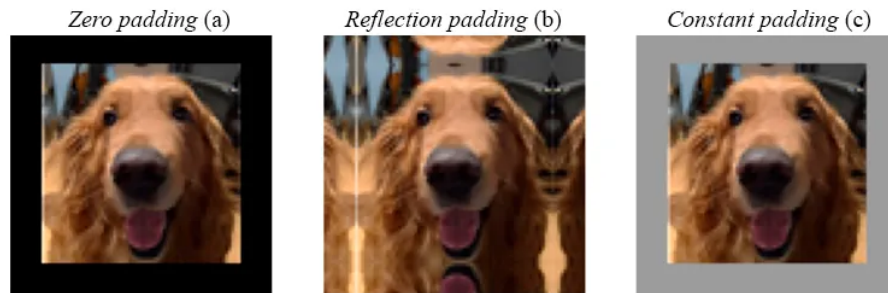
1.3.3 Padding

Durante a aplicação da convolução, é comum que a dimensão espacial (largura e altura) da imagem de saída seja menor do que a da imagem de entrada. Isso ocorre porque o *kernel* não pode ser aplicado nas bordas da imagem sem ultrapassar seus limites. Para mitigar essa redução de dimensão, uma técnica chamada *padding* é frequentemente utilizada. O padding envolve adicionar uma borda de pixels ao redor da imagem original antes de aplicar a convolução. Esses pixels adicionais podem ser preenchidos com zeros (*zero-padding*) ou com valores replicados dos pixels mais próximos da borda da imagem original, (*reflection padding*). O uso de padding permite que o *kernel* seja aplicado em todas as regiões da imagem, incluindo as bordas, preservando assim as dimensões espaciais da imagem de entrada na saída.

O cálculo da largura e da altura da imagem de saída com padding pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= w_{in} - w_{kernel} + 2w_{pad} + 1, \\ h_{out} &= h_{in} - h_{kernel} + 2h_{pad} + 1, \end{aligned} \tag{1.24}$$

Figura 1.19: Aplicação de padding em uma imagem. Em (a) a imagem original passou por zero padding. Em (b) a imagem original passou por *reflection padding*. A imagem em (c) passou por um padding constante, semelhante em (a), mas com valor diferente de zero.



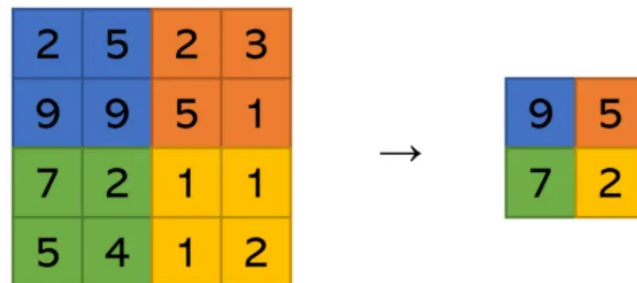
Fonte: Adaptado de [Vinícius Trevisan 2021].

onde w_{pad} e h_{pad} são a quantidade de pixels adicionados como padding na largura e na altura, respectivamente.

1.3.4 Pooling

O pooling é uma operação utilizada em redes neurais convolucionais para reduzir as dimensões espaciais (largura e altura) dos mapas de características, mantendo as informações mais relevantes. Essa redução ajuda a diminuir a complexidade computacional da rede, além de tornar o modelo mais robusto a pequenas variações na entrada, como translações e rotações. Existem diferentes tipos de operações de pooling, sendo as mais comuns o *Max Pooling* e o *Average Pooling*. A técnica de pooling substitui agrupamentos de pixels por um valor que os represente. A figura 1.20 ilustra o funcionamento do max pooling de tamanho 2, em que os pixels são agrupados em quadrados de tamanho 2x2 e são substituídos pelo maior valor de intensidade de pixel do grupo.

Figura 1.20: Exemplo de Max Pooling com tamanho de 2x2. A imagem original é reduzida para a imagem menor após a aplicação do Max Pooling. Cada bloco de 2x2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida.



Fonte: Adaptado de [Vinícius Trevisan 2021].

Como consequência imediata a aplicação do *pooling*, a informação fica mais condensada, o que pode levar a uma perda de detalhes finos na imagem. No entanto, essa perda é compensada pela capacidade do modelo de focar nas características mais importantes e gerais da imagem,

melhorando sua capacidade de generalização em tarefas de reconhecimento de padrões.

1.3.5 Stride

O *stride* (passo) é um parâmetro utilizado nas operações de convolução e pooling em redes neurais convolucionais que determina o número de pixels que o *kernel* de pooling se desloca ao longo da imagem de entrada. Em outras palavras, o stride define o quanto o *kernel* avance a cada aplicação da operação. Um stride de 1 significa que o *kernel* se mova um pixel por vez, enquanto um stride maior que 1 faz com que o *kernel* pule pixels, resultando em uma redução mais rápida das dimensões espaciais da imagem. O uso do stride afeta diretamente o tamanho da saída gerada pela operação de convolução ou pooling. O cálculo da largura e da altura da imagem de saída com stride pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= \frac{w_{in} - w_{kernel} + 2w_{pad}}{s_w} + 1, \\ h_{out} &= \frac{h_{in} - h_{kernel} + 2h_{pad}}{s_h} + 1, \end{aligned} \tag{1.25}$$

onde s_w e s_h são os valores do stride na largura e na altura, respectivamente.

1.3.6 Convolução Transposta

A convolução transposta, também conhecida como deconvolução ou convolução fracionária, é uma operação utilizada em redes neurais convolucionais para aumentar as dimensões espaciais (largura e altura) dos mapas de características. Essa operação é frequentemente empregada em tarefas de geração de imagens, onde o objetivo é reconstruir imagens a partir de representações compactas. A convolução transposta funciona de maneira inversa à convolução tradicional. Enquanto a convolução reduz as dimensões espaciais da imagem de entrada, a convolução transposta expande essas dimensões. Isso é alcançado aplicando o *kernel* de convolução de forma que ele “espalhe” os valores dos pixels da imagem de entrada para posições mais amplas na imagem de saída. A Figura 1.21 ilustra o processo pelo qual uma imagem 3×3 passa por uma convolução transposta de *kernel* 3×3 e stride 1 para gerar uma imagem 5×5 .

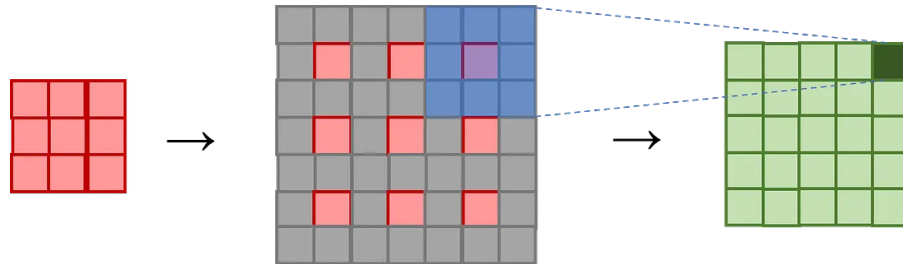
O cálculo da largura e da altura da imagem de saída com convolução transposta pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= (w_{in} - 1) \cdot s_w - 2w_{pad} + w_{kernel}, \\ h_{out} &= (h_{in} - 1) \cdot s_h - 2h_{pad} + h_{kernel}, \end{aligned} \tag{1.26}$$

onde s_w e s_h são os valores do stride na largura e na altura, respectivamente.

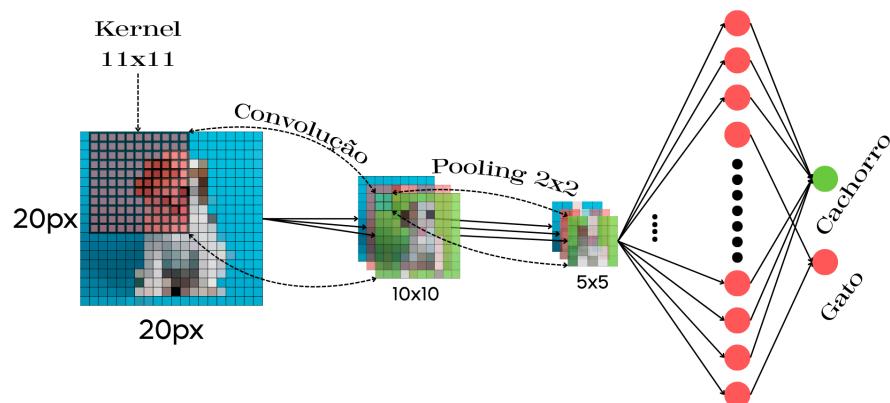
Exemplo 1.6 A figura 1.22 apresenta uma arquitetura típica de uma Rede Neural Convolucional (CNN).

Figura 1.21: Convolução transposta. Uma convolução de *kernel* 3×3 com stride 1 em uma imagem 5×5 geraria uma imagem de dimensões 3×3 . A convolução transposta aplicada na imagem 3×3 (vermelha) para se obter uma imagem 5×5 (verde) é feita em duas etapas. Primeiramente (centro) se insere pixels de valor 0, representados pela cor cinza, entre os pixels da imagem original, juntamente com pixels de borda. Em seguida se faz uma convolução nessa imagem alterada também usando um *kernel* 3×3 e stride 1.



Fonte: Adaptado de [Vinícius Trevisan 2021].

Figura 1.22: Arquitetura típica de uma Rede Neural Convolutiva com uma camada de convolução e uma camada de pooling, em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem.



Fonte: Elaboração do Autor.

1.4 U-net

Modificando a arquitetura tradicional de uma CNN, é possível obter variadas redes neurais com objetivos específicos. O uso típico de redes convolucionais ocorre em tarefas de classificação, onde o objetivo é atribuir uma etiqueta a uma imagem inteira. No entanto, em muitas tarefas visuais, especialmente no processamento de imagens biomédicas, a saída desejada deve incluir informações espaciais detalhadas, como a localização precisa de estruturas dentro da imagem. Para atender a essa necessidade, foi proposta a arquitetura U-Net [Ronneberger, Fischer e Brox 2015], que é uma rede neural convolucional projetada para segmentação de imagens. A U-Net é composta por uma parte de contração (encoder) e uma parte de expansão (decoder), formando uma estrutura em forma de U. O encoder é responsável por extrair características da imagem de entrada, enquanto o decoder reconstrói a imagem segmentada, utilizando as características extraídas pelo encoder. A arquitetura U-Net é especialmente eficaz em tarefas de segmentação semântica, onde o objetivo é classificar cada pixel da imagem em uma categoria específica, permitindo a identificação precisa de objetos e estruturas dentro da imagem.

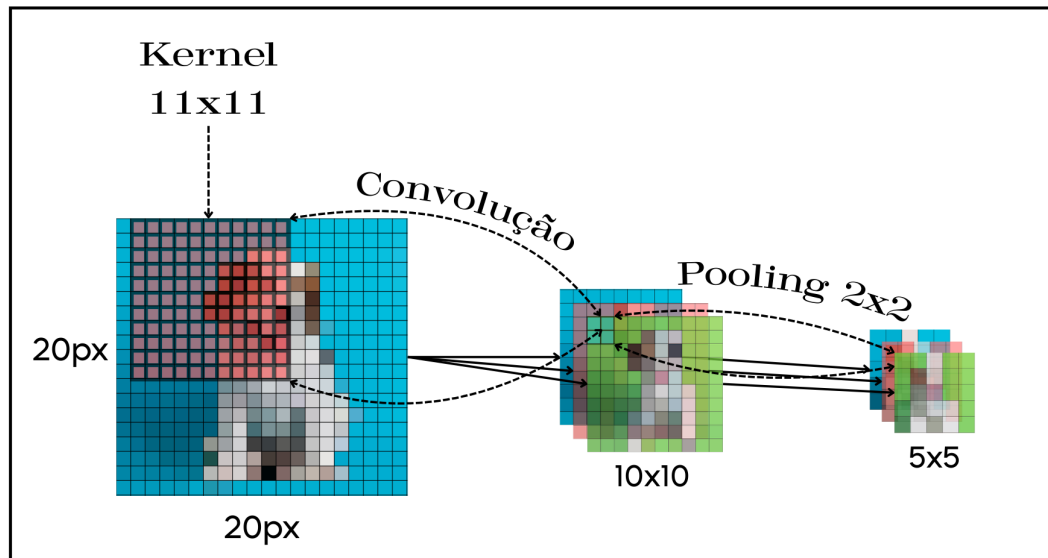
Assim como no Exemplo 1.22, teríamos a parte de convolução e pooling para extrair as características da imagem e logo depois a parte de expansão para reconstruir a imagem segmentada, como podemos ver na Figura 1.23.

1.4.1 Copy and Crop

Uma característica importante da arquitetura U-Net é a presença de conexões de atalho (skip connections) entre as camadas correspondentes do encoder e do decoder. Essas conexões permitem que as informações de baixo nível, como bordas e texturas, sejam diretamente transferidas do encoder para o decoder, facilitando a reconstrução precisa da imagem segmentada. O processo de copiar e recortar (copy and crop) envolve copiar as características extraídas pelo encoder e recortá-las para se encaixar nas dimensões da camada correspondente no decoder. Isso é feito para garantir que as informações relevantes sejam preservadas durante a reconstrução da imagem segmentada, melhorando a precisão da segmentação, especialmente em regiões onde os detalhes são cruciais.

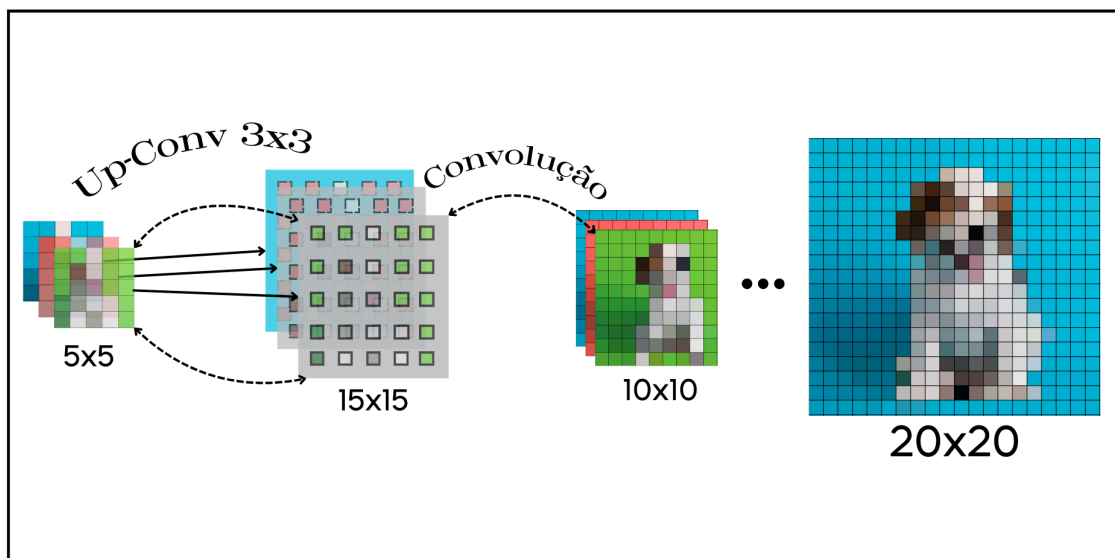
Figura 1.23: Arquitetura da U-Net, composta por uma parte de contração (encoder) e uma parte de expansão (decoder). O encoder extrai características da imagem de entrada, enquanto o decoder reconstrói a imagem segmentada.

Bloco Encoder



(a) Encoder da U-Net, onde as setas indicam as operações de convolução e pooling que extraem características da imagem de entrada.

Bloco Decoder



(b) Decoder da U-Net que reconstrói a imagem segmentada utilizando as características extraídas pelo encoder.

Fonte: Elaboração do Autor.

Referências Bibliográficas

- [Bear, Connors e Paradiso 2017]BEAR, M. F.; CONNORS, B. W.; PARADISO, M. A. *Neurociências: desvendando o sistema nervoso*. 4. ed. Porto Alegre: Artmed, 2017. ISBN 9788582713091.
- [Bengio 2012]BENGIO, Y. Practical recommendations for gradient-based training of deep architectures. *Cornell University*, n. Version 2, 2012.
- [Crevier 1993]CREVIER, D. *AI: The Tumultuous History of the Search for Artificial Intelligence*. New York: Basic Books, 1993. ISBN 0-465-02997-3.
- [Deep Learning Book 2025]Deep Learning Book. *Capítulo 19: Overfitting e Regularização: Parte 1*. 2025. Acessado: 2026-01-26. Disponível em: <<https://www.deeplearningbook.com.br/overfitting-e-regularizacao-parte-1/>>.
- [Hecht-Nielsen 1992]HECHT-NIELSEN, R. Theory of the backpropagation neural network. *Academic Press*, 1992.
- [McCulloch 1943]MCCULLOCH, W. P. W. S. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 1943.
- [Müller 2008]MÜLLER, V. C. Review of margaret boden, "mind as machine: A history of cognitive science". *Minds and Machines*, Springer, v. 18, n. 1, p. 121–125, 2008.
- [Ronneberger, Fischer e Brox 2015]RONNEBERGER, O.; FISCHER, P.; BROX, T. U-net: Convolutional networks for biomedical image segmentation. In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Munich, Germany: Springer, 2015. p. 234–241.
- [Rosenblatt 1958]ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958.
- [Ruder 2016]RUDER, S. An overview of gradient descent optimization algorithms. *Insight Centre for Data Analytics, NUI Galway Aylien Ltd., Dublin*, 2016.
- [Stanfield 2014]STANFIELD, C. L. *Fisiologia Humana*. [S.l.]: Pearson Education do Brasil, 2014.

- [Sutskever 2013]SUTSKEVER, I. On the importance of initialization and momentum in deep learning. *International Conference on Machine Learning*, 2013.
- [Szeliski 2010]SZELISKI, R. *Computer Vision: Algorithms and Applications*. [S.l.]: Springer, 2010.
- [Vinícius Trevisan 2021]Vinícius Trevisan. *Como funcionam as Redes Neurais Convolucionais (CNNs)*. 2021. Acessado: 2025-12-16. Disponível em: <<https://medium.com/data-hackers/como-funcionam-as-redes-neurais-convolucionais-cnns-71978185c1>>.
- [Werbos 1990]WERBOS, P. J. Backpropagation through time: What it does and how to do it. *PROCEEDINGS OF THE IEEE*, 1990.
- [Yann LeCun 1998]Yann LeCun. *The MNIST Database of Handwritten Digits*. 1998. Acessado: 2026-02-22. Disponível em: <<http://yann.lecun.com/exdb/mnist/>>.